



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA0211 – C Programming

to the award of the degree of

**BACHELOR OF ENGINEERING
IN
ELECTRONICS AND COMMUNICATION ENGINEERING
AND
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

Y. Hema Sai Lahari (192311446)

Manasa G (192525315)

Kaviyaasri M (192312614)

Under the Supervision of

Dr. Saravanan S K

**SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “ **SECURE PHARMACY INVENTORY AND PRESCRIPTION MANAGEMENT SYSTEM WITH MEDICINE STOCK TRACKING**” has been carried out by **KAVIYAASRI M (192312614)**, **Y HEMA SAI LAHARI (192311446)**, **MANASA G (192525315)** under the supervision of **DR SARAVANAN S K** and is submitted in partial fulfilment of the requirements of the course CSA0211 – C PROGRAMMING for the B.E. Electronics and Communication Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

Submitted for the Project Work Viva-Voce held on _____.

COURSE FACULTY COORDINATOR

Dr. Anusuya

Program Director

CSE

Saveetha School of Engineering

SIMATS

COURSE

Dr. Saravanan S K

Professor

CSE

Saveetha School of Engineering

SIMATS

INTERNAL EXAMINER

EXTERNAL EXAMINER

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, **KAVIYAASRI M (192312614)**, **MANASA G(192525315)**, **HEMA SAI LAHARI Y (192311446)** of the Department of Electronics and Communication Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**SECURE PHARMACY INVENTORY AND PRESCRIPTION MANAGEMENT SYSTEM WITH MEDICINE STOCK TRACKING**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed.

We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

Y. Hema Sai Lahari (192311446)

Manasa G (192525315)

Kaviyaasri M (192312614)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing / Analysis Contribution	Report Contribution	Approx. Contribution (%)
HEMA SAI LAHARI Y (192311446)	Module 1 (Medicine Inventory Management): medicine record creation, medicine details management, medicine search, and inventory handling	Developed functions for adding, viewing, searching, and updating medicine records using Structures, Arrays, Strings, and File Handling	Tested medicine addition, searching, updating, and retrieval using different medicine records	Module 1 documentation, algorithm, implementation details, and test cases	33.33%
KAVIYAASRIM (192312614),	Module 2 (Prescription & Patient Medicine Management): patient record management, prescription creation, medicine selection, dosage and quantity management	Developed patient and prescription functions using Structures, Arrays, Strings, and File Handling for adding, viewing, searching, and updating prescription records	Tested patient and prescription operations using different patient details, medicines, dosages, and quantities; verified correct record storage and retrieval	Module 2 documentation, algorithm, implementation details, test cases, and output analysis	33.33%
MANASA G(192525315)	Module 3 (Stock & Expiry Monitoring): medicine stock tracking, quantity monitoring, expiry-date management, and stock-status checking	Developed functions for updating medicine stock quantities and checking expiry and stock status	Tested stock updates, expiry-date checking, low-stock conditions, and medicine status using different test records	Module 3 documentation, testing results, output analysis, and conclusion	33.34%

Total: 100%

ABSTRACT

Pharmacy management requires accurate handling of medicine inventory, patient details, prescriptions, stock levels, and expiry information. Manual management of these records can lead to data entry errors, difficulty in tracking medicine availability, and inefficient handling of patient prescriptions. A computerized pharmacy management system can reduce these difficulties by providing an organized method for storing, retrieving, and updating pharmacy-related information.

This project presents the design and implementation of a **Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking** using the C programming language. The system is developed to manage medicine inventory, patient prescriptions, medicine quantities, stock levels, and expiry dates efficiently. The application uses Structures, Arrays, Strings, and File Handling to store and manage records systematically. The prescription and patient medicine management module allows patient details to be recorded, prescriptions to be created, medicines to be associated with patients, and dosage and quantity information to be maintained. The system also supports searching and retrieving stored records when required.

The system was tested using different patient details, medicine records, prescription information, stock quantities, and expiry conditions. Results show that the application can store and retrieve pharmacy records accurately, manage patient prescriptions efficiently, and monitor medicine stock and expiry information with reduced manual effort. The project demonstrates how C programming concepts can be applied to develop a simple, secure, and reliable pharmacy management solution and provides a foundation for future extensions such as database integration, graphical user interfaces, automated alerts, and cloud-based record management.

TABLE OF CONTENTS

S.No.	Title	Page No.
1	Bonafide Certificate	2
2	Declaration	3
3	Individual Contribution Statement	4
4	Abstract	5
5	List of Figures	8
6	List of Tables	8
7	List of Abbreviations / Symbols	9
8	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	10-12
9	1.1 Background	10
10	1.2 Need for the Project	10
11	1.3 Problem Statement	11
12	1.4 Project Aim	11
13	1.5 Project Objectives	11
14	1.6 Scope	12
15	1.7 Expected Engineering Outcome	12
16	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	13-14
17	2.1 Review Method	13
18	2.2 Review of Existing Work	13
19	2.3 Comparative Analysis	14
20	2.4 Research / Engineering Gap	14
21	2.5 Proposed Contribution	14
22	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	15-21
23	3.1 Requirements	15
24	3.2 Constraints & Applicable Standards	16
25	3.3 Design Specifications	16
26	3.4 Alternative Solutions & Evaluation	16
27	3.5 Selected Design & Detailed Design	17
28	3.6 Trade-off Analysis	18
29	3.7 Sustainability, Ethics, Safety, Risk & Security	20
30	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	22-25
31	4.1 Development Approach & Resources	22
32	4.2 Hardware / Software Implementation & Modern Tools Used	22
33	4.3 Test Plan & Verification	22
34	4.4 Results	23
35	4.5 Data Analysis & Engineering Judgment	24
36	4.6 Comparison with Existing Solutions & Achievement of Objectives	24
37	4.7 Strengths & Limitations	24
38	4.8 Project Planning, Roles & Budget	25

39	CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	26-27
40	5.1 Conclusion	26
41	5.2 Future Work	26
42	5.3 Professional Learning & Individual Reflection	26
43	REFERENCES	28
44	APPENDICES	29-31
55	Capstone Project Outcome Mapping Sheet	32

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1.1	System Architecture of the Proposed Obstacle Detection System	18
Figure 2.1	Decision Logic Flow for SAFE / WARNING / STOP Zones	20

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Comparative Analysis of Existing Obstacle-Detection Approaches	14
Table 2	Stakeholder / User Requirements	15
Table 3	Functional & Non-Functional Requirements	15
Table 4	Constraints & Applicable Standards	16
Table 5	Design Specifications	16
Table 6	Alternative Solutions - Evaluation Matrix	17
Table 7	Trade-off Analysis	20
Table 8	Sustainability, Ethics, Safety & Security Evaluation	20
Table 9	Tools and Technologies Used	22
Table 10	Sample Test Cases and Results	22

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
RF	Radio Frequency	—
mmWave	Millimetre Wave (radar)	—
cm	Centimetre	cm
LED	Light Emitting Diode	—
I/O	Input / Output	—
AV	Autonomous Vehicle	—
LiDAR	Light Detection and Ranging	—
TC	Test Case	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Modern pharmacy operations require accurate and organized management of medicine records, patient information, prescriptions, stock quantities and expiry details. Among the different activities performed in a pharmacy, prescription and patient medicine management is important for maintaining correct medication information and ensuring that prescribed medicines are properly recorded. Manual handling of these records can result in data-entry mistakes, loss of information and difficulty in retrieving previous prescription details. This project investigates the use of a C-based pharmacy management system to provide a structured and reliable method for managing patient prescriptions and medicine information.

The prescription and patient medicine management module maintains patient details and records the medicines prescribed to each patient. The system uses C programming concepts such as Structures, Arrays, Strings and File Handling to organize and store the required information. Patient details can be entered and retrieved when required, while prescription records can be created with medicine names, dosage and quantity information. The stored prescription information complements the medicine inventory module by associating available medicines with individual patient prescriptions. The resulting records can therefore be used to support organized prescription management and efficient pharmacy operations.

1.2 Need for the Module

- Manual prescription management can result in incorrect or incomplete patient and medicine records.
- A structured patient record is required to maintain prescription information accurately and retrieve it when needed.
- Recording medicine name, dosage and quantity is necessary for maintaining complete prescription details.
- File-based storage provides a simple and low-cost method for preserving patient and prescription records between program executions.
- Searching and retrieving prescription information enables the pharmacy to access previously stored patient medicine details efficiently.

1.3 Problem Statement

A pharmacy management system must be able to maintain accurate patient and prescription information and provide quick access to the medicines associated with each patient. Manual record keeping can make it difficult to store, search and update prescription details, particularly when multiple patient records are maintained. Without an organized prescription-management mechanism, medicine name, dosage and quantity information may be incorrectly recorded or difficult to retrieve. The engineering problem addressed by this module is therefore to implement a C-based patient and prescription management system

using Structures, Arrays, Strings and File Handling that can create patient records, store prescription details, associate medicines with patients, and retrieve or update the required information efficiently.

1.4 Module Aim

To design and implement a reliable patient and prescription management module using C programming to maintain patient medicine records, prescription details, dosage and quantity information in an organized manner.

1.5 Module Objectives

- To study and apply C programming concepts for patient and prescription management.
- To create and maintain structured patient records using C Structures and Arrays.
- To record prescription details including medicine name, dosage and quantity.
- To store and retrieve patient and prescription information using File Handling.
- To provide efficient searching and updating of stored prescription records.
- To test the prescription and patient medicine management module using different patient and medicine records.

1.6 Scope

In-Scope

- Patient record management using the C programming language.
- Creation and maintenance of prescription records for individual patients.
- Recording medicine names, dosage and prescribed quantities.
- Storage and retrieval of patient and prescription information using files.
- Searching and updating stored patient and prescription records.

Out-of-Scope

- Diagnosis of diseases or recommendation of medicines.
- Automatic generation of prescriptions by medical professionals.
- Online doctor consultation or electronic medical-record integration.
- Online pharmacy payment or medicine delivery services.
- Integration with external hospital databases or cloud-based healthcare systems.

1.7 Expected Engineering Outcome

The expected outcome of this module is a working C-based patient and prescription management subsystem comprising structured patient records, prescription information and medicine details. The module will allow patient information to be entered, prescription records to be created, and medicine name, dosage and quantity to be stored and retrieved using file-based data management. The resulting prescription data will provide organized information for the overall pharmacy management system. Collectively, the module will demonstrate a simple, reliable and efficient method of managing patient medicine records while reducing manual effort and improving the accuracy and accessibility of prescription information, as presented in the subsequent chapters.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature was identified from research papers, journal articles and technical studies covering computerized pharmacy management, electronic prescription systems, patient medicine records and inventory management using software-based solutions. Five key areas were reviewed and compared based on the data-management approach used, record-handling capability, storage method, limitations and relevance to a low-cost C-based pharmacy management system. Particular attention was given to structured patient and prescription management techniques that can provide organized record storage and retrieval with simple programming concepts and low computational requirements.

2.2 Review of Existing Work

Existing pharmacy management systems have demonstrated the usefulness of computerized solutions for managing medicine records, patient information and prescriptions. Digital systems allow pharmacy personnel to store, retrieve and update records more efficiently than manual registers. Patient prescription systems can maintain information such as patient details, medicine names, dosage and prescribed quantity, making the required information easier to access when needed. These approaches provide a basic foundation for developing a structured prescription and patient medicine management module.

Several studies have investigated electronic prescription and patient-record systems, showing that computerized storage can reduce repetitive manual work and improve the accessibility of healthcare-related information. However, larger systems may require databases, network connectivity, advanced security mechanisms and greater implementation complexity. For educational and prototype-level applications, a simpler file-based approach can provide the essential record management functions while requiring fewer computational and hardware resources.

Research on integrated pharmacy management systems further demonstrates the advantage of connecting patient prescriptions with medicine information. A patient record can be associated with the medicines prescribed, including dosage and quantity, while inventory information can be maintained separately for stock monitoring. This concept is directly relevant to the proposed system, where the prescription and patient medicine module provides organized patient and prescription information that can complement the medicine inventory and stock-monitoring modules. The stored records can then be searched, retrieved and updated when required.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual prescription records	Simple to implement and does not require computer hardware	Time-consuming, difficult to search, and prone to data-entry and record-loss errors	Provides a reference for identifying the need for computerized patient and prescription management
Spreadsheet-based pharmacy records	Easy to create, edit and organize basic pharmacy information	Limited automation, greater possibility of accidental modification and weak record relationships	Demonstrates the benefit of moving toward a structured C-based record-management system
Database-based pharmacy systems	Provides structured storage, searching, updating and management of large records	Requires database software, greater implementation complexity and additional configuration	Represents an advanced alternative to the simple file-based approach
Electronic prescription systems	Improves prescription accessibility and reduces manual paperwork	May require network infrastructure, database systems and additional security mechanisms	Provides the basis for maintaining digital patient and prescription information
C-based file management system	Low cost, simple implementation and suitable for structured record storage and retrieval	Limited scalability compared with full database systems	Forms the proposed patient and prescription management approach using C programming

Table 1: Comparative Analysis of Existing Pharmacy Record and Prescription Management Approaches

2.4 Research / Engineering Gap

The reviewed literature shows that computerized pharmacy and electronic prescription systems can improve record accessibility, reduce manual work and organize patient medicine information more effectively than traditional paper-based methods. However, large-scale solutions often require database platforms, network connectivity and higher implementation complexity. There remains a need for a simple, low-cost implementation that demonstrates the fundamental principles of patient record management, prescription handling and medicine information storage using basic programming concepts. Such an approach is particularly useful for educational and prototype-level pharmacy applications.

2.5 Proposed Contribution

This project contributes a low-cost C-based patient and prescription management module using Structures, Arrays, Strings and File Handling to provide organized storage and retrieval of patient medicine information. The module records patient details, prescription information, medicine names, dosage and quantities, and supports searching and updating of stored records. While simpler than database-driven pharmacy systems described in existing work, the proposed module demonstrates the fundamental engineering principles of structured data management and file-based record handling. It provides the prescription and patient medicine information required to support the overall pharmacy inventory and stock-tracking system.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Pharmacy staff	A simple and organized method for creating, viewing, searching and updating patient prescription records
Patients	Accurate maintenance of prescribed medicine, dosage and quantity information
Educational institutions	A demonstrable C programming example for patient records, prescription management and file-based data storage
Future system developers	A modular prescription-management component that can later be integrated with database or healthcare management systems

Table 2: Stakeholder / User Requirements

Functional & Non-Functional Requirements

Type	Description	Measurable Target
Functional	Create and store patient records	Patient details stored correctly for each registered patient
Functional	Create prescription records for patients	Prescription information associated correctly with the selected patient
Functional	Store medicine name, dosage and quantity	All prescription fields recorded accurately
Functional	Search and retrieve patient or prescription records	Correct matching records displayed for valid search inputs
Functional	Update existing patient and prescription information	Updated information stored and retrieved correctly
Non-Functional	Response time	Record search and retrieval completed within a short processing time
Non-Functional	Reliability	Stored patient and prescription records retrieved without data loss during normal operation
Non-Functional	Usability	Menu-driven interface clearly displays available patient and prescription operations

Table 3: Functional & Non-Functional Requirements

3.2 Constraints & Applicable Standards

As a C-based educational and prototype-level application, the project is primarily constrained by available development time, system resources and the limitations of file-based data storage. The system is designed for local pharmacy record management and does not provide direct integration with external healthcare databases or online services.

Standard / Guideline	Requirement	How Applied in This Project
C programming conventions	Consistent and readable program structure, variable naming and function organization	Followed throughout the patient, prescription and record-management functions
Structured project development lifecycle	Requirement analysis → design → implementation → testing → evaluation	Used as the project's development process
Modular software design	Independent functions with clear responsibilities and data flow	Patient record, prescription management, searching and updating functions are organized separately
File-handling practices	Proper storage and retrieval of records from files	File operations are used to maintain patient and prescription information between program executions

Table 4: Constraints & Applicable Standards

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Patient record capacity	Multiple patient records can be maintained	Records	High
Prescription record storage	Prescription details associated with patients	Records	High
Medicine information	Medicine name, dosage and quantity stored correctly	—	High
Search operation	Retrieve patient and prescription information using valid identifiers	—	High
Update operation	Modify existing patient or prescription information	—	High
Data storage	Records retained using file handling	File	High
Record retrieval	Correct stored information displayed when requested	—	High
Program usability	Menu-driven operations for simple record management	—	Medium

Table 5: Design Specifications

3.4 Alternative Solutions & Evaluation

Alternative 1 – Manual record system: Patient and prescription information is maintained using paper registers without computerized storage.

Alternative 2 – Database-based system: Patient and prescription records are stored using a relational database with structured queries and database-management functions.

Alternative 3 (Selected) – C-based file management system: Patient and prescription information is managed using C Structures, Arrays, Strings and File Handling, providing a simple and low-cost computerized solution.

Criterion	Weight	Manual Records	Database System	C File-Based System (Selected)
Record accessibility	25%	Low	High	High
Implementation complexity	20%	Low	High	Medium
Data storage capability	20%	Low	High	Medium-High
Search and update functionality	20%	Low	High	High
Cost and resource requirement	15%	Low	Medium-High	Low
Overall suitability	100%	Low	High	High

Table 6: Alternative Solutions - Evaluation Matrix

3.5 Selected Design & Detailed Design

Alternative 3 (C-based file management system) was selected because it provides the required patient and prescription management functions with low implementation complexity and minimal resource requirements. It supports structured storage of patient details, prescription information, medicine names, dosage and quantities while providing searching and updating operations. The system follows a modular design in which each function performs a specific record-management task.

The Module 2 design consists of the following major functional stages:

- **Patient Record Management** - Creates and maintains patient information using C Structures and Arrays.
- **Prescription Creation** - Records prescription details and associates prescribed medicines with the corresponding patient.
- **Medicine & Dosage Management** - Stores medicine name, dosage and prescribed quantity as part of the prescription record.
- **Search & Retrieval** - Searches stored patient and prescription records and displays the required information.

- **Update & File Management** - Updates existing records and uses File Handling to store and retrieve information for later use.

The selected design provides a simple, organized and reliable approach for managing patient medicine and prescription information. It also allows the module to interact with the other modules of the pharmacy management system, particularly the medicine inventory and stock-monitoring functions.

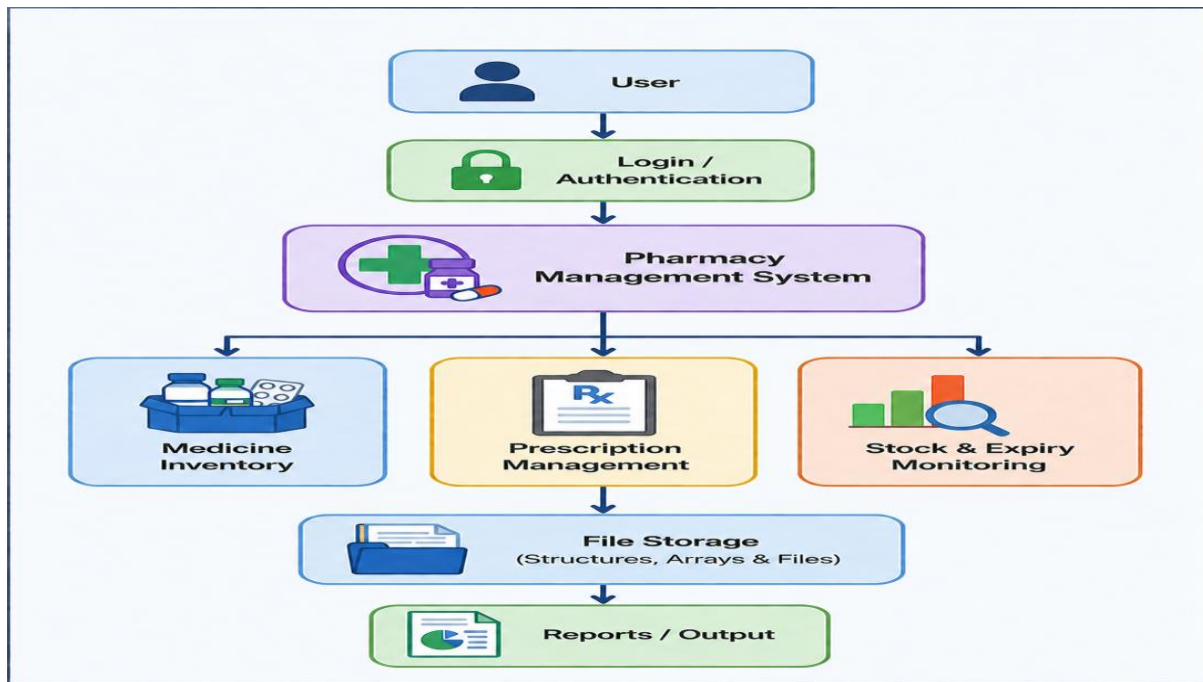


Figure 1.1: System Architecture

MODULE 2: Prescription & Patient Medicine Management (Kaviyaasri M)

Module Objectives

- To create and maintain patient records using C Structures and Arrays.
- To record prescription details including medicine name, dosage and quantity.
- To store and retrieve patient and prescription information using File Handling.
- To provide searching and updating facilities for stored patient and prescription records.
- To associate prescribed medicines with the corresponding patient records.
- To improve the accuracy and organization of pharmacy prescription management.

Problem Addressed

The pharmacy system requires an organized method for maintaining patient and prescription information. Manual record keeping can result in incomplete patient details, incorrect prescription information and difficulty in retrieving previous records. Module 2 addresses this problem by providing a computerized C-based mechanism for creating patient records,

storing prescription details and associating medicines, dosage and quantities with individual patients. The module also supports searching and updating of stored records so that prescription information can be accessed efficiently when required.

Work Performed

- Studied the requirements for patient and prescription record management.
- Designed patient and prescription data structures using C Structures.
- Implemented functions for adding and storing patient information.
- Created prescription records containing medicine name, dosage and quantity.
- Associated prescription information with the corresponding patient records.
- Implemented searching and retrieval of patient and prescription records.
- Implemented updating of existing patient and prescription information.
- Used File Handling to store records and retrieve them during subsequent program executions.
- Tested the module using different patient details, medicines, dosages and quantities.
- Verified that the stored information was correctly displayed and updated during testing.

Individual Contribution

As the lead contributor to Module 2 (33.33% overall project contribution), **Kaviyaasri M** was responsible for designing and implementing patient and prescription record management functions, developing C Structures and Arrays for organized data storage, implementing File Handling for record persistence, testing patient and prescription operations using different input records, and documenting the design, implementation and testing results of the module.

Methodology / Approach

The patient and prescription management module uses a structured record-management approach implemented in the C programming language. Patient information is first entered through the menu-driven interface and stored using C Structures. Prescription information is then created for the selected patient, including medicine name, dosage and prescribed quantity.

The system uses Arrays to manage multiple records and Strings to store textual information such as patient names and medicine names. File Handling is used to write records into files and retrieve them whenever required. Search operations identify the required patient or prescription record, while update operations modify existing information and save the changes.

The overall process can be represented as:

Patient Details → Patient Record → Prescription Creation → Medicine, Dosage & Quantity → File Storage → Search / Retrieval / Update

The resulting prescription information can be used together with the medicine inventory and stock-monitoring modules to support the overall pharmacy management system.

Expected Outcome

A reliable patient and prescription management module that can create, store, search and update patient medicine records. The module is expected to maintain accurate medicine names, dosage and quantity information while providing organized access to prescription records through file-based storage.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Patient record storage	Separate variables for each patient	Simple for a small number of records	Difficult to manage multiple patients and increases code complexity	Structures selected – provides organized storage of related patient information
Prescription data storage	Individual variables for each prescription field	Easy to understand initially	Difficult to scale and maintain multiple prescription records	Structures and Arrays selected – allow multiple prescription records to be managed systematically
Data storage method	Temporary memory-only storage	Simple and fast during program execution	Records are lost when the program terminates	File Handling selected – preserves patient and prescription records between executions
Record retrieval	Manual browsing of all records	Easy to implement	Time-consuming when many records are present	Search function selected – provides faster access to required patient and prescription information
Record modification	Delete and recreate the complete record	Simple implementation approach	Can result in unnecessary data entry and record duplication	Update function selected – allows existing information to be modified directly
User interaction	Command-line menu system	Low cost and simple to implement	Less visual compared with a graphical interface	Menu-driven C interface selected – sufficient for the prototype and educational requirements

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Low-resource software implementation suitable for educational and small-scale pharmacy applications	C programming requires limited computational resources and the system does not require continuous network connectivity	Lightweight C-based implementation retained for the prototype
Ethics	Patient and prescription information must be handled responsibly and should not be modified or displayed unnecessarily	Considered the importance of maintaining accurate patient and medicine records	Structured records and controlled operations used to reduce incorrect data handling
Health & Safety	Incorrect prescription information such as medicine name, dosage or quantity may lead to inappropriate medicine information being displayed	Tested different patient, medicine, dosage and quantity inputs	Input verification and record checking included to improve information accuracy
Data Security	Patient and prescription records should not be unnecessarily exposed to external systems	The current prototype uses local file storage without external network connectivity	Local file-based storage retained to minimize external access and communication risks
Data Integrity	Incorrect updates or duplicate records may reduce the reliability of stored information	Tested record creation, searching, retrieval and updating using different inputs	Structured record management and validation used to maintain consistent information

Table 9: Sustainability, Ethics, Safety & Security Evaluation – Module 2

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation
Incorrect patient information entered during record creation	Medium	Medium	Medium	Validate input values and verify patient information before storing the record
Incorrect medicine, dosage or quantity entered in a prescription	Medium	High	Medium-High	Check prescription fields during entry and verify the stored record through testing
Patient or prescription record cannot be retrieved correctly	Low	High	Medium	Test file creation, storage and retrieval operations using multiple records
File corruption or accidental loss of stored records	Low	High	Medium	Use proper file-opening, reading and writing operations and maintain backup copies during testing
Duplicate patient or prescription records	Medium	Medium	Medium	Search existing records before creating or updating records where required
Incorrect update modifies the wrong record	Low	High	Medium	Use appropriate patient identifiers and verify the selected record before updating
Unauthorized access to locally stored records	Low	High	Medium	Keep files within the controlled application environment and avoid unnecessary external access
Program error during data entry or file operation	Medium	Medium	Medium	Perform repeated test cases using valid and invalid inputs and verify program output

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed following a structured process of requirement analysis, modular software design, implementation, testing and evaluation, with each of the three modules developed and tested according to its assigned functionality before integration into the complete pharmacy management system. Development was carried out using the C programming language and fundamental programming concepts such as Structures, Arrays, Strings, Functions and File Handling. The project used commonly available computer resources and did not require paid software, external datasets or cloud-based services for the prototype implementation.

4.2 Software Implementation & Modern Tools Used

S.No.	Component	Technology / Method Used	Purpose
1	Programming Language	C Programming	Development of the complete pharmacy management system
2	Data Management	Structures and Arrays	Organized storage and management of medicine, patient and prescription records
3	String Processing	C Strings	Storage and handling of patient names, medicine names and other textual information
4	File Management	C File Handling	Permanent storage and retrieval of pharmacy records
5	User Interface	Menu-driven Console Interface	Provides options for adding, viewing, searching and updating records
6	Development Environment	C Compiler / IDE	Writing, compiling, debugging and executing the program
7	Testing & Debugging	Test Cases and Program Output	Verification of record creation, searching, updating and retrieval functions

Table 10: Tools and Technologies Used

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Patient record creation	Enter valid patient details and create a new patient record (TC01)	Patient information stored and displayed correctly
Prescription creation	Enter prescription details for a registered patient (TC02)	Prescription record created and associated with the correct patient
Medicine information storage	Enter medicine name, dosage and quantity (TC03)	All medicine-related prescription details stored correctly
Patient search	Search for an existing patient using the appropriate identifier (TC04)	Correct patient record displayed
Prescription search	Search for an existing prescription record (TC05)	Correct prescription and medicine details displayed
Record update	Modify an existing patient or prescription record (TC06)	Updated information stored and displayed correctly
File retrieval	Close and reopen the program and retrieve stored records (TC07)	Previously stored records remain available and accurate

Table 11: Sample Test Cases and Results

4.4 Results

All seven test cases were used to verify the major functions of the patient and prescription management module. The results were compared with the design specifications defined in Chapter 3.

Specification	Required Value	Achieved Value	Status
Patient record creation (TC01)	Patient details stored correctly	Patient details stored and displayed correctly	Pass
Prescription creation (TC02)	Prescription associated with patient	Prescription associated correctly	Pass
Medicine information (TC03)	Medicine, dosage and quantity recorded	Required details recorded correctly	Pass
Patient search (TC04)	Correct patient record retrieved	Matching patient record displayed	Pass
Prescription search (TC05)	Correct prescription retrieved	Matching prescription details displayed	Pass
Record update (TC06)	Existing information modified correctly	Updated information displayed correctly	Pass
File retrieval (TC07)	Stored records available after program restart	Records retrieved correctly from file	Pass

Table 12: Test Results

A representative test run involved entering patient information followed by a prescription containing medicine name, dosage and quantity. The stored patient and prescription records were then searched and retrieved through the menu-driven interface. An update operation was subsequently performed to verify that existing information could be modified correctly. The results demonstrated that the module was able to create, store, retrieve and update patient and prescription information as required.

4.5 Data Analysis & Engineering Judgment

The testing results demonstrated that the patient and prescription management module performed the required record-management operations correctly. Patient information and prescription details were stored using structured data representations, while File Handling provided persistent storage between program executions. Search and retrieval operations allowed previously stored records to be accessed efficiently, and update operations enabled existing information to be modified without recreating the complete record.

During testing, particular attention was given to the correctness of medicine names, dosage values and prescribed quantities because these fields form an important part of prescription information. Different patient and prescription records were used to verify that records remained correctly associated and that the required information was displayed during retrieval. The use of modular functions and structured data improved the organization and maintainability of the implementation.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Study and apply C programming concepts for pharmacy management	Use of Structures, Arrays, Strings, Functions and File Handling	Fully Achieved
Create and maintain patient records	Patient record creation and retrieval functions	Fully Achieved
Record prescription details including medicine, dosage and quantity	Prescription creation and medicine-detail management	Fully Achieved
Store and retrieve records using File Handling	File writing, reading and retrieval tests	Fully Achieved
Provide searching and updating of stored records	Search and update test cases	Fully Achieved
Test the module using different patient and medicine records	TC01–TC07 using different input records	Fully Achieved
Integrate the module with the complete pharmacy management system	Module 2 provides patient and prescription information for interaction with inventory and stock-monitoring modules	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Simple and low-cost implementation using the C programming language.
- Organized management of patient and prescription information using Structures and Arrays.
- File-based storage allows records to remain available between program executions.
- Searching and updating functions improve the accessibility and management of stored records.
- Medicine name, dosage and quantity can be maintained as part of individual prescription records.

Limitations

- The system uses file-based storage and does not provide a full database-management system.
- The current implementation is primarily a console-based application without a graphical user interface.
- The system does not provide online doctor consultation or external healthcare-system integration.
- Advanced authentication, encryption and cloud-based synchronization are outside the current project scope.

4.8 Project Planning and Roles

Development proceeded over four major milestones: (1) requirement analysis and literature review, (2) design and implementation of the individual pharmacy management modules, (3) integration and testing of patient, prescription, inventory and stock-related functions, and (4) final testing, documentation and evaluation.

Student	Assigned Responsibility	Actual Contribution
Hema Sai	Module 1: Medicine Inventory Management	Implemented and tested medicine record creation, viewing, searching and inventory-management functions
Kaviyaasri M (192312614)	Module 2: Prescription & Patient Medicine Management	Implemented and tested patient records, prescription creation, medicine details, searching, updating and file-based record management
Manasa	Module 3: Stock & Expiry Monitoring	Implemented and tested medicine stock quantity tracking, expiry-date management and stock-status checking

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project set out to design and implement a low-cost pharmacy inventory and prescription management system for maintaining medicine, patient, prescription, stock and expiry information, and this aim was achieved. The system uses the C programming language with Structures, Arrays, Strings and File Handling to provide organized management of pharmacy records. Module 2 focuses on patient and prescription management by creating patient records and associating prescribed medicines with medicine names, dosage and quantity information. Evaluation using the defined test cases confirmed that the system can create, store, search, retrieve and update patient and prescription records correctly. The project demonstrates how fundamental C programming concepts can be applied to develop a simple, reliable and organized pharmacy management solution.

5.2 Future Work

- Integrate a database management system such as MySQL or SQLite for improved storage capacity and scalability.
- Develop a graphical user interface to provide a more user-friendly method of managing patient, prescription and medicine records.
- Add user authentication and role-based access control to provide stronger protection for patient and prescription information.
- Implement automated alerts for low medicine stock, approaching expiry dates and prescription-related requirements.
- Integrate the system with hospital or healthcare databases for secure sharing and synchronization of patient prescription information.
- Extend the system with cloud-based storage and backup facilities to provide remote access and improved data recovery.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical understanding of using C Structures and Arrays to organize patient and prescription records.
- Techniques for using Strings to store and manage patient names, medicine names and other textual information.

- Understanding of File Handling for storing, retrieving and maintaining pharmacy records between program executions.

Engineering & Professional Skills Developed

- Design and implementation of modular patient and prescription management functions using C programming.
- Testing, debugging and verification of patient records, medicine information, dosage and quantity using defined test cases.
- Team-based software development with clearly divided and individually accountable module responsibilities.

Individual Reflection - Kaviyaasri M (192312614)

The most challenging part of Module 2 was designing the patient and prescription records so that medicine information, dosage and quantity were correctly associated with the corresponding patient. This was addressed by using C Structures and Arrays to organize related information and by testing different patient and prescription records during implementation. A key decision I made was to use File Handling so that patient and prescription information could be stored permanently and retrieved when required, instead of keeping the records only during program execution. Working on this module gave me a much clearer, hands-on understanding of structured data management, file-based storage and modular programming in C. If repeating the project, I would explore database integration and stronger validation and authentication mechanisms to improve the scalability, security and reliability of the pharmacy management system.

REFERENCES

- [1] S. Karimi, Y. Jahani, and L. Ahmadian, “The effect of electronic prescription systems on pharmacy performance through evaluation of existing infrastructure in Kerman Iran,” *Scientific Reports*, vol. 15, 2025.
- [2] Y. Zheng, J. Gong, and C. A. Lester, “Evaluating the impact of pharmacy-led RxChange interventions on medication use and cost outcomes for electronic prescriptions,” *Journal of the American Pharmacists Association*, vol. 65, no. 3, 2025.
- [3] J. Gong, V. D. Marshall, M. Whitaker, et al., “Enhancing medication safety with System Approach to Verifying Electronic Prescriptions (SAV E-Rx): pharmacists’ review of product selection outcomes between prescribed and dispensed medications,” *BMJ Health & Care Informatics*, vol. 32, 2025.
- [4] A. Lambarth, D. Wainwright, T. Saha, et al., “Electronic medicine management systems in developing countries: A landscape review,” *British Journal of Clinical Pharmacology*, vol. 91, no. 9, pp. 2507–2514, 2025.
- [5] “Evaluation of an electronic prescription platform: Clinicians’ feedback on three distinct services aiming to facilitate clinical decision and safer e-prescription,” *Research in Social and Administrative Pharmacy*, vol. 20, no. 7, pp. 640–647, 2024.
- [6] P. Babbrah, M. R. Solomon, L. A. Stember, J. W. Hill, and M. Weiker, “Formulary & benefit and real-time pharmacy Benefit: Electronic standards delivering value to prescribers and pharmacists,” *Journal of the American Pharmacists Association*, vol. 63, no. 3, pp. 725–730, 2023.
- [7] J. Li, C. Lester, X. Zhao, Y. Ding, Y. Jiang, and V. G. Vinod Vydiswaran, “PharmMT: A Neural Machine Translation Approach to Simplify Prescription Directions,” 2022.
- [8] T. T. Nguyen, H. D. Nguyen, T. H. Nguyen, H. H. Pham, I. Ide, and P. L. Nguyen, “A Novel Approach for Pill-Prescription Matching with GNN Assistance and Contrastive Learning,” 2022.
- [9] “The impact of introducing automated dispensing cabinets, barcode medication administration, and closed-loop electronic medication management systems on work processes and safety of controlled medications in hospitals: A systematic review,” *Research in Social and Administrative Pharmacy*, vol. 17, no. 5, pp. 832–841, 2021.
- [10] “Detecting Potential Medication Selection Errors During Outpatient Pharmacy Processing of Electronic Prescriptions With the RxNorm Application Programming Interface: Retrospective Observational Cohort Study,” *JMIR Medical Informatics*, vol. 8, no. 3, 2020.

APPENDICES

APPENDIX A – Prescription Management Decision Logic Table

Operation	Input / Condition	System Action	Output
Add Patient	Valid patient details entered	Create patient record	Patient record stored
Create Prescription	Registered patient selected	Enter medicine, dosage and quantity	Prescription created
Search Patient	Valid patient identifier entered	Search stored records	Matching patient details displayed
Search Prescription	Valid prescription/patient identifier entered	Search prescription records	Matching prescription displayed
Update Record	Existing record selected	Modify required information	Updated record stored
Invalid Record	Patient or prescription not found	Display error message	Record not found

APPENDIX B – Working Principle (Step Sequence)

- **1. Patient Entry** – The user enters the required patient details through the menu-driven interface.
- **2. Record Creation** – The entered patient information is organized using C Structures and stored as a patient record.
- **3. Prescription Entry** – A prescription is created for the selected patient by entering the medicine name, dosage and prescribed quantity.
- **4. Record Storage** – The patient and prescription information is stored using Arrays and File Handling for future retrieval.
- **5. Search** – The system searches the stored records using the appropriate patient or prescription identifier.
- **6. Update** – When required, the selected patient or prescription information is modified and the updated record is stored.
- **7. Retrieval** – The system displays the requested patient and prescription information through the menu-driven interface.

APPENDIX C – Sample Program Output

===== PHARMACY MANAGEMENT SYSTEM =====

1. Add Patient
2. Create Prescription
3. Search Patient
4. Search Prescription
5. Update Record
6. Exit

Enter your choice: 1

Enter Patient ID: P101

Enter Patient Name: Arun Kumar

Enter Age: 45

Patient record added successfully.

Enter your choice: 2

Enter Patient ID: P101

Enter Medicine Name: Paracetamol

Enter Dosage: 500 mg

Enter Quantity: 10

Prescription added successfully.

Enter your choice: 3

Enter Patient ID: P101

Patient ID: P101

Patient Name: Arun Kumar

Age: 45

Patient record found successfully.

Enter your choice: 4

Enter Patient ID: P101

Medicine: Paracetamol

Dosage: 500 mg

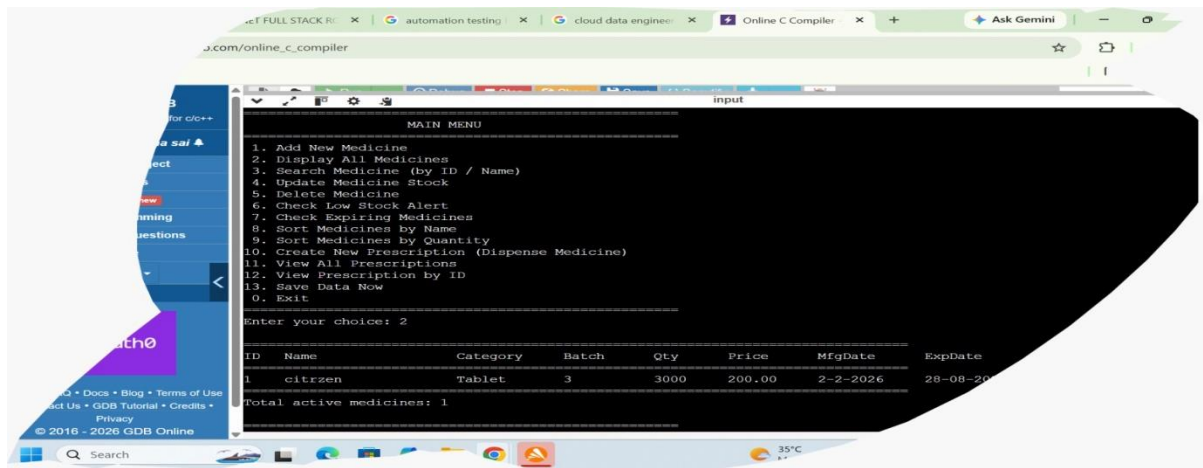
Quantity: 10

Prescription record found successfully.

Enter your choice: 5

Enter Patient ID: P101

Record updated successfully.



```

=====
--- ADMIN LOGIN ---
Username: admin
Password: admin123

Login successful! Welcome, admin.

=====
MAIN MENU
=====
1. Add New Medicine
2. Display All Medicines
3. Search Medicine (by ID / Name)
4. Update Medicine Stock
5. Delete Medicine
6. Check Low Stock Alert
7. Check Expiring Medicines
8. Sort Medicines by Name
9. Sort Medicines by Quantity
10. Create New Prescription (Dispense Medicine)
11. View All Prescriptions
12. View Prescription by ID
13. Save Data Now
0. Exit
=====
Enter your choice: 1

```

```

=====
0. Exit
=====
Enter your choice: 1

-- ADD NEW MEDICINE (ID: 1) --
Medicine Name: citrzen
Manufacturer: shashi
Batch Number: 3
Category (0-Tablet 1-Syrup 2-Injection 3-Capsule 4-Ointment 5-Drops 6-Other): 0
Quantity in Stock: 3000
Unit Price (Rs.): 200
Manufacturing Date (DD-MM-YYYY): 2-2-2026
Expiry Date (DD-MM-YYYY): 28-08-2028

Medicine 'citrzen ' added successfully with ID 1!

=====
MAIN MENU
=====
1. Add New Medicine
2. Display All Medicines
3. Search Medicine (by ID / Name)
4. Update Medicine Stock
5. Delete Medicine
6. Check Low Stock Alert
7. Check Expiring Medicines
8. Sort Medicines by Name
9. Sort Medicines by Quantity

```

```

10. Create New Prescription (Dispense Medicine)
11. View All Prescriptions
12. View Prescription by ID
13. Save Data Now
0. Exit
=====
Enter your choice: 1

-- ADD NEW MEDICINE (ID: 2) --
Medicine Name: pp
Manufacturer: kl
Batch Number: 3
Category (0-Tablet 1-Syrup 2-Injection 3-Capsule 4-Ointment 5-Drops 6-Other): 4
Quantity in Stock: 299
Unit Price (Rs.): 26
Manufacturing Date (DD-MM-YYYY): 12-09-1999
Expiry Date (DD-MM-YYYY): 20-09-2028

Medicine 'pp' added successfully with ID 2!

=====
MAIN MENU
=====
1. Add New Medicine

```

APPENDIX D – CODE

```
/*
=====
=====
SECURE PHARMACY INVENTORY AND PRESCRIPTION MANAGEMENT SYSTEM
WITH MEDICINE STOCK TRACKING
=====
=====
Concepts demonstrated:
- Structures (Medicine, Prescription, PrescribedItem, Admin)
- Arrays of structures (inventory[], prescriptions[])
- Functions (modular design, 25+ functions)
- File Handling (binary files for persistent storage)
- String handling (string.h)
- Login/Authentication ("Secure" system access)
- Searching (linear search by ID / name)
- Sorting (bubble sort by name / quantity)
- Enumerations (Category)
- Pointers (passed to functions by reference)
- Control structures (switch-case menu, loops)
=====
=====
Compile : gcc pharmacy_management.c -o pharmacy
Run    : ./pharmacy
Default login -> Username: admin Password: admin123
=====
=====
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
/* ----- CONSTANTS ----- */
#define MAX_MEDICINES 200
#define MAX_PRESCRIPTIONS 200
#define MAX_ITEMS_PER_PRESCRIPTION 10
#define MEDICINE_FILE "medicines.dat"
#define PRESCRIPTION_FILE "prescriptions.dat"
#define LOW_STOCK_THRESHOLD 10
#define ADMIN_USER "admin"
#define ADMIN_PASS "admin123"
/* ----- ENUMS ----- */
typedef enum {
    TABLET,
    SYRUP,
    INJECTION,
    CAPSULE,
    OINTMENT,
    DROPS,
    OTHER} Category;
```

```

const char *categoryNames[] = {
    "Tablet", "Syrup", "Injection", "Capsule", "Ointment", "Drops", "Other"
};

/* ----- STRUCTURES ----- */
typedef struct {
    int id;
    char name[50];
    char manufacturer[50];
    char batchNo[20];
    Category category;
    int quantity;
    float unitPrice;
    char mfgDate[12]; /* DD-MM-YYYY */
    char expiryDate[12]; /* DD-MM-YYYY */
    int isActive; /* 1 = active, 0 = deleted */
} Medicine;
typedef struct {
    int medicineId;
    char medicineName[50];
    int quantity;
    float subTotal;
} PrescribedItem;

typedef struct {
    int prescriptionId;
    char patientName[50];
    int patientAge;
    char doctorName[50];
    char date[12];
    PrescribedItem items[MAX_ITEMS_PER_PRESCRIPTION];
    int itemCount;
    float totalAmount;
} Prescription;

/* ----- GLOBAL ARRAYS ----- */
Medicine inventory[MAX_MEDICINES];
Prescription prescriptions[MAX_PRESCRIPTIONS];
int medicineCount = 0;
int prescriptionCount = 0;
int nextMedicineId = 1;
int nextPrescriptionId = 1001;

/* ----- FUNCTION PROTOTYPES ----- */
int loginScreen();
void mainMenu();
/* Inventory operations */
void addMedicine();
void displayAllMedicines();
void searchMedicineMenu();
int findMedicineIndexById(int id);

```

```

int findMedicineIndexByName(const char name[]);
void updateMedicineStock();
void deleteMedicine();
void checkLowStock();
void checkExpiringMedicines();
void sortMedicinesByName();
void sortMedicinesByQuantity();
void printMedicineRow(Medicine m);
/* Prescription operations */
void createPrescription();
void viewAllPrescriptions();
void viewPrescriptionById();
void printPrescription(Prescription p);
/* File handling */
void saveMedicinesToFile();
void loadMedicinesFromFile();
void savePrescriptionsToFile();
void loadPrescriptionsFromFile();
/* Utility */
void clearInputBuffer();
void toLowerStr(char *str);
void pause_screen();
/*
=====
=====
MAIN FUNCTION
=====
===== */
int main() {
    loadMedicinesFromFile();
    loadPrescriptionsFromFile();
    printf("=====\\n");
    printf(" SECURE PHARMACY INVENTORY & PRESCRIPTION SYSTEM\\n");
    printf("=====\\n");
    if (!loginScreen()) {
        printf("\\nToo many failed login attempts. Exiting for security.\\n");
        return 0;
    }
    mainMenu();
    /* Persist data before exit */
    saveMedicinesToFile();
    savePrescriptionsToFile();
    printf("\\nAll data saved securely. Goodbye!\\n");
    return 0;
}
/*
=====
=====
LOGIN / SECURITY

```

```

=====
===== */
int loginScreen() {
    char user[30], pass[30];
    int attempts = 3;

    while (attempts > 0) {
        printf("\n--- ADMIN LOGIN ---\n");
        printf("Username: ");
        scanf("%29s", user);
        printf("Password: ");
        scanf("%29s", pass);

        if (strcmp(user, ADMIN_USER) == 0 && strcmp(pass, ADMIN_PASS) == 0) {
            printf("\nLogin successful! Welcome, %s.\n", user);
            return 1;
        } else {
            attempts--;
            printf("Invalid credentials. Attempts left: %d\n", attempts);
        }
    }
    return 0;
}

/*
=====
=====
MAIN MENU

=====
===== */
void mainMenu() {
    int choice;
    do {
        printf("\n=====
        printf("          MAIN MENU\n");
        printf("=====
        printf(" 1. Add New Medicine\n");
        printf(" 2. Display All Medicines\n");
        printf(" 3. Search Medicine (by ID / Name)\n");
        printf(" 4. Update Medicine Stock\n");
        printf(" 5. Delete Medicine\n");
        printf(" 6. Check Low Stock Alert\n");
        printf(" 7. Check Expiring Medicines\n");
        printf(" 8. Sort Medicines by Name\n");
        printf(" 9. Sort Medicines by Quantity\n");
        printf("10. Create New Prescription (Dispense Medicine)\n");
        printf("11. View All Prescriptions\n");
        printf("12. View Prescription by ID\n");
    } while (choice != 0);
}

```

```

printf("13. Save Data Now\n");
printf(" 0. Exit\n");
printf("=====\n");
printf("Enter your choice: ");

if (scanf("%d", &choice) != 1) {
    clearInputBuffer();
    choice = -1;
}
switch (choice) {
    case 1: addMedicine(); break;
    case 2: displayAllMedicines(); break;
    case 3: searchMedicineMenu(); break;
    case 4: updateMedicineStock(); break;
    case 5: deleteMedicine(); break;
    case 6: checkLowStock(); break;
    case 7: checkExpiringMedicines(); break;
    case 8: sortMedicinesByName(); break;
    case 9: sortMedicinesByQuantity(); break;
    case 10: createPrescription(); break;
    case 11: viewAllPrescriptions(); break;
    case 12: viewPrescriptionById(); break;
    case 13:
        saveMedicinesToFile();
        savePrescriptionsToFile();
        printf("\nData saved successfully.\n");
        break;
    case 0: printf("\nExiting system...\n"); break;
    default: printf("\nInvalid choice. Please try again.\n");
}
} while (choice != 0);
}
/*
=====
=====
INVENTORY: ADD MEDICINE
=====
===== */
void addMedicine() {
    if (medicineCount >= MAX_MEDICINES) {
        printf("\nInventory full! Cannot add more medicines.\n");
        return;
    }
    Medicine m;
    int catChoice;
    clearInputBuffer();
    m.id = nextMedicineId++;
    printf("\n--- ADD NEW MEDICINE (ID: %d) ---\n", m.id);
    printf("Medicine Name: ");
    fgets(m.name, sizeof(m.name), stdin);

```

```

m.name[strcspn(m.name, "\n")] = '\0';
printf("Manufacturer: ");
fgets(m.manufacturer, sizeof(m.manufacturer), stdin);
m.manufacturer[strcspn(m.manufacturer, "\n")] = '\0';
printf("Batch Number: ");
fgets(m.batchNo, sizeof(m.batchNo), stdin);
m.batchNo[strcspn(m.batchNo, "\n")] = '\0';
printf("Category (0-Tablet 1-Syrup 2-Injection 3-Capsule 4-Ointment 5-Drops 6-Other):
");
scanf("%d", &catChoice);
if (catChoice < 0 || catChoice > 6) catChoice = 6;
m.category = (Category)catChoice;
printf("Quantity in Stock: ");
scanf("%d", &m.quantity);
printf("Unit Price (Rs.): ");
scanf("%f", &m.unitPrice);
clearInputBuffer();
printf("Manufacturing Date (DD-MM-YYYY): ");
fgets(m.mfgDate, sizeof(m.mfgDate), stdin);
m.mfgDate[strcspn(m.mfgDate, "\n")] = '\0';
printf("Expiry Date (DD-MM-YYYY): ");
fgets(m.expiryDate, sizeof(m.expiryDate), stdin);
m.expiryDate[strcspn(m.expiryDate, "\n")] = '\0';
m.isActive = 1;
inventory[medicineCount++] = m;
saveMedicinesToFile();
printf("\nMedicine '%s' added successfully with ID %d!\n", m.name, m.id);
}
/*
=====
=====
INVENTORY: DISPLAY ALL
=====
===== */
void printMedicineRow(Medicine m) {
    printf("%-4d %-20.19s %-12.11s %-10.9s %-8d %-10.2f %-12s %-12s\n",
        m.id, m.name, categoryNames[m.category], m.batchNo,
        m.quantity, m.unitPrice, m.mfgDate, m.expiryDate);
}
void displayAllMedicines() {
    int found = 0;
    printf("\n=====
=====
===== \n");
    printf("%-4s %-20s %-12s %-10s %-8s %-10s %-12s %-12s\n",
        "ID", "Name", "Category", "Batch", "Qty", "Price", "MfgDate", "ExpDate");
    printf("=====
===== \n");
    for (int i = 0; i < medicineCount; i++) {
        if (inventory[i].isActive) {
            printMedicineRow(inventory[i]);

```

```

        found = 1;
    }
}

if (!found) printf("No medicines found in inventory.\n");

printf("=====\n");
printf("Total active medicines: %d\n", medicineCount);
}

/*
=====
=====
INVENTORY: SEARCH
=====
===== */
int findMedicineIndexById(int id) {
    for (int i = 0; i < medicineCount; i++) {
        if (inventory[i].id == id && inventory[i].isActive) return i;
    }
    return -1;
}

int findMedicineIndexByName(const char name[]) {
    char searchName[50], itemName[50];
    strcpy(searchName, name);
    toLowerStr(searchName);

    for (int i = 0; i < medicineCount; i++) {
        if (!inventory[i].isActive) continue;
        strcpy(itemName, inventory[i].name);
        toLowerStr(itemName);
        if (strstr(itemName, searchName) != NULL) return i;
    }
    return -1;
}

void searchMedicineMenu() {
    int choice;
    printf("\nSearch by: 1. ID 2. Name\nChoice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        int id;
        printf("Enter Medicine ID: ");
        scanf("%d", &id);
        int idx = findMedicineIndexById(id);
        if (idx == -1) {

```



```

        printf("\nMedicine with ID %d not found.\n", id);
    } else {
        printf("\n--- MEDICINE FOUND ---\n");
        printf("%-4s %-20s %-12s %-10s %-8s %-10s %-12s %-12s\n",
            "ID", "Name", "Category", "Batch", "Qty", "Price", "MfgDate", "ExpDate");
        printMedicineRow(inventory[idx]);
    }
} else if (choice == 2) {
    char name[50];
    clearInputBuffer();
    printf("Enter Medicine Name (or part of it): ");
    fgets(name, sizeof(name), stdin);
    name[strcspn(name, "\n")] = '\0';

    int idx = findMedicineIndexByName(name);
    if (idx == -1) {
        printf("\nNo medicine matching '%s' found.\n", name);
    } else {
        printf("\n--- MEDICINE FOUND ---\n");
        printf("%-4s %-20s %-12s %-10s %-8s %-10s %-12s %-12s\n",
            "ID", "Name", "Category", "Batch", "Qty", "Price", "MfgDate", "ExpDate");
        printMedicineRow(inventory[idx]);
    }
} else {
    printf("\nInvalid choice.\n");
}
}

/*
=====
=====
INVENTORY: UPDATE STOCK
=====
===== */
void updateMedicineStock() {
    int id, qty, mode;
    printf("\nEnter Medicine ID to update: ");
    scanf("%d", &id);

    int idx = findMedicineIndexById(id);
    if (idx == -1) {
        printf("\nMedicine not found.\n");
        return;
    }

    printf("Current stock of '%s' = %d\n", inventory[idx].name, inventory[idx].quantity);
    printf("1. Add Stock  2. Reduce Stock  3. Set Exact Quantity\nChoice: ");
    scanf("%d", &mode);
    printf("Enter quantity: ");

```

```

scanf("%d", &qty);

switch (mode) {
    case 1: inventory[idx].quantity += qty; break;
    case 2:
        if (qty > inventory[idx].quantity) {
            printf("\nCannot reduce more than available stock!\n");
            return;
        }
        inventory[idx].quantity -= qty;
        break;
    case 3: inventory[idx].quantity = qty; break;
    default: printf("\nInvalid choice.\n"); return;
}

saveMedicinesToFile();
printf("\nStock updated! New quantity of '%s' = %d\n", inventory[idx].name,
inventory[idx].quantity);
}

/*
=====
=====
INVENTORY: DELETE (soft delete)

=====
===== */
void deleteMedicine() {
    int id;
    printf("\nEnter Medicine ID to delete: ");
    scanf("%d", &id);

    int idx = findMedicineIndexById(id);
    if (idx == -1) {
        printf("\nMedicine not found.\n");
        return;
    }

    char confirm;
    printf("Are you sure you want to delete '%s'? (y/n): ", inventory[idx].name);
    scanf(" %c", &confirm);

    if (tolower(confirm) == 'y') {
        inventory[idx].isActive = 0;
        saveMedicinesToFile();
        printf("\nMedicine '%s' deleted successfully.\n", inventory[idx].name);
    } else {
        printf("\nDeletion cancelled.\n");
    }
}
}

```

```

/*
=====
=====
INVENTORY: LOW STOCK / EXPIRY ALERTS

=====
===== */
void checkLowStock() {
    int found = 0;
    printf("\n--- LOW STOCK ALERT (Threshold: %d) ---\n",
LOW_STOCK_THRESHOLD);
    printf("%-4s %-20s %-8s\n", "ID", "Name", "Qty");
    for (int i = 0; i < medicineCount; i++) {
        if (inventory[i].isActive && inventory[i].quantity < LOW_STOCK_THRESHOLD) {
            printf("%-4d %-20.19s %-8d\n", inventory[i].id, inventory[i].name,
inventory[i].quantity);
            found = 1;
        }
    }
    if (!found) printf("No medicines are low on stock.\n");
}

void checkExpiringMedicines() {
    /* Simplified check: compares expiry year/month string prefix entered by user */
    char cutoff[12];
    printf("\nEnter cutoff date to check expiry before (DD-MM-YYYY): ");
    clearInputBuffer();
    fgets(cutoff, sizeof(cutoff), stdin);
    cutoff[strcspn(cutoff, "\n")] = '\0';

    int found = 0;
    printf("\n--- MEDICINES EXPIRING BEFORE %s (string compare on YYYY-MM-DD
basis) ---\n", cutoff);
    printf("Note: Enter dates consistently as DD-MM-YYYY for meaningful comparison.\n");
    printf("%-4s %-20s %-12s\n", "ID", "Name", "ExpDate");

    for (int i = 0; i < medicineCount; i++) {
        if (!inventory[i].isActive) continue;
        /* Reformat DD-MM-YYYY to YYYY-MM-DD for correct string comparison */
        char expReformat[12], cutReformat[12];
        sscanf(inventory[i].expiryDate, "%2s-%2s-%4s",
            &expReformat[6], &expReformat[3], expReformat);
        sscanf(cutoff, "%2s-%2s-%4s",
            &cutReformat[6], &cutReformat[3], cutReformat);
        expReformat[4] = '-'; expReformat[7] = '-';
        cutReformat[4] = '-'; cutReformat[7] = '-';
        expReformat[10] = '\0'; cutReformat[10] = '\0';

        if (strcmp(expReformat, cutReformat) < 0) {

```

```

        printf("%-4d %-20.19s %-12s\n", inventory[i].id, inventory[i].name,
inventory[i].expiryDate);
        found = 1;
    }
}
if (!found) printf("No medicines expiring before the given date.\n");
}

/*
=====
=====
INVENTORY: SORTING (Bubble Sort)

=====
===== */
void sortMedicinesByName() {
    Medicine temp;
    for (int i = 0; i < medicineCount - 1; i++) {
        for (int j = 0; j < medicineCount - i - 1; j++) {
            if (strcasecmp(inventory[j].name, inventory[j + 1].name) > 0) {
                temp = inventory[j];
                inventory[j] = inventory[j + 1];
                inventory[j + 1] = temp;
            }
        }
    }
    printf("\nMedicines sorted by name.\n");
    displayAllMedicines();
    saveMedicinesToFile();
}

void sortMedicinesByQuantity() {
    Medicine temp;
    for (int i = 0; i < medicineCount - 1; i++) {
        for (int j = 0; j < medicineCount - i - 1; j++) {
            if (inventory[j].quantity > inventory[j + 1].quantity) {
                temp = inventory[j];
                inventory[j] = inventory[j + 1];
                inventory[j + 1] = temp;
            }
        }
    }
    printf("\nMedicines sorted by quantity (ascending).\n");
    displayAllMedicines();
    saveMedicinesToFile();
}

/*
=====
=====

```

PRESCRIPTION: CREATE (also reduces medicine stock -> dispensing)

```
=====
===== */
void createPrescription() {
    if (prescriptionCount >= MAX_PRESCRIPTIONS) {
        printf("\nPrescription records full!\n");
        return;
    }

    Prescription p;
    p.prescriptionId = nextPrescriptionId++;
    p.itemCount = 0;
    p.totalAmount = 0.0f;

    clearInputBuffer();
    printf("\n--- CREATE PRESCRIPTION (ID: %d) ---\n", p.prescriptionId);

    printf("Patient Name: ");
    fgets(p.patientName, sizeof(p.patientName), stdin);
    p.patientName[strcspn(p.patientName, "\n")] = '\0';

    printf("Patient Age: ");
    scanf("%d", &p.patientAge);

    clearInputBuffer();
    printf("Doctor Name: ");
    fgets(p.doctorName, sizeof(p.doctorName), stdin);
    p.doctorName[strcspn(p.doctorName, "\n")] = '\0';

    printf("Date (DD-MM-YYYY): ");
    fgets(p.date, sizeof(p.date), stdin);
    p.date[strcspn(p.date, "\n")] = '\0';

    int addMore = 1;
    while (addMore && p.itemCount < MAX_ITEMS_PER_PRESCRIPTION) {
        int id, qty;
        printf("\nEnter Medicine ID to prescribe: ");
        scanf("%d", &id);

        int idx = findMedicineIndexById(id);
        if (idx == -1) {
            printf("Medicine not found. Try again.\n");
            continue;
        }

        printf("Available stock: %d. Enter quantity to prescribe: ", inventory[idx].quantity);
        scanf("%d", &qty);

        if (qty <= 0 || qty > inventory[idx].quantity) {
```

```

        printf("Invalid quantity or insufficient stock!\n");
        continue;
    }

    /* Dispense: reduce stock */
    inventory[idx].quantity -= qty;

    PrescribedItem item;
    item.medicineId = inventory[idx].id;
    strcpy(item.medicineName, inventory[idx].name);
    item.quantity = qty;
    item.subTotal = qty * inventory[idx].unitPrice;

    p.items[p.itemCount++] = item;
    p.totalAmount += item.subTotal;

    printf("Added %d x %s to prescription. Subtotal: Rs.%.2f\n",
        qty, item.medicineName, item.subTotal);

    clearInputBuffer();
    printf("Add another medicine? (y/n): ");
    char c;
    scanf(" %c", &c);
    addMore = (tolower(c) == 'y');
}

if (p.itemCount == 0) {
    printf("\nNo medicines added. Prescription cancelled.\n");
    nextPrescriptionId--;
    return;
}

prescriptions[prescriptionCount++] = p;
saveMedicinesToFile();
savePrescriptionsToFile();

printf("\n--- PRESCRIPTION CREATED SUCCESSFULLY ---\n");
printPrescription(p);
}

/*
=====
=====
    PRESCRIPTION: VIEW

=====
===== */
void printPrescription(Prescription p) {

```

```

printf("\n=====\\n")
;
printf("Prescription ID : %d\\n", p.prescriptionId);
printf("Patient      : %s (Age: %d)\\n", p.patientName, p.patientAge);
printf("Doctor       : %s\\n", p.doctorName);
printf("Date        : %s\\n", p.date);
printf("-----\\n");
printf("%-6s %-20s %-6s %-10s\\n", "MedID", "Medicine", "Qty", "SubTotal");
for (int i = 0; i < p.itemCount; i++) {
    printf("%-6d %-20.19s %-6d Rs.%.2f\\n",
        p.items[i].medicineId, p.items[i].medicineName,
        p.items[i].quantity, p.items[i].subTotal);
}
printf("-----\\n");
printf("TOTAL AMOUNT   : Rs.%.2f\\n", p.totalAmount);

printf("=====\\n");
}

```

```

void viewAllPrescriptions() {
    if (prescriptionCount == 0) {
        printf("\\nNo prescriptions recorded yet.\\n");
        return;
    }
    for (int i = 0; i < prescriptionCount; i++) {
        printPrescription(prescriptions[i]);
    }
}

```

```

void viewPrescriptionById() {
    int id;
    printf("\\nEnter Prescription ID: ");
    scanf("%d", &id);

    for (int i = 0; i < prescriptionCount; i++) {
        if (prescriptions[i].prescriptionId == id) {
            printPrescription(prescriptions[i]);
            return;
        }
    }
    printf("\\nPrescription with ID %d not found.\\n", id);
}

```

```

/*

```

```

=====
=====

```

FILE HANDLING (Binary persistence)

```

=====
===== */
void saveMedicinesToFile() {
    FILE *fp = fopen(MEDICINE_FILE, "wb");
    if (fp == NULL) {
        printf("\nError: Could not save medicine data.\n");
        return;
    }
    fwrite(&medicineCount, sizeof(int), 1, fp);
    fwrite(&nextMedicineId, sizeof(int), 1, fp);
    fwrite(inventory, sizeof(Medicine), medicineCount, fp);
    fclose(fp);
}

void loadMedicinesFromFile() {
    FILE *fp = fopen(MEDICINE_FILE, "rb");
    if (fp == NULL) return; /* No existing data yet */

    fread(&medicineCount, sizeof(int), 1, fp);
    fread(&nextMedicineId, sizeof(int), 1, fp);
    fread(inventory, sizeof(Medicine), medicineCount, fp);
    fclose(fp);
}

void savePrescriptionsToFile() {
    FILE *fp = fopen(PRESCRIPTION_FILE, "wb");
    if (fp == NULL) {
        printf("\nError: Could not save prescription data.\n");
        return;
    }
    fwrite(&prescriptionCount, sizeof(int), 1, fp);
    fwrite(&nextPrescriptionId, sizeof(int), 1, fp);
    fwrite(prescriptions, sizeof(Prescription), prescriptionCount, fp);
    fclose(fp);
}

void loadPrescriptionsFromFile() {
    FILE *fp = fopen(PRESCRIPTION_FILE, "rb");
    if (fp == NULL) return;

    fread(&prescriptionCount, sizeof(int), 1, fp);
    fread(&nextPrescriptionId, sizeof(int), 1, fp);
    fread(prescriptions, sizeof(Prescription), prescriptionCount, fp);
    fclose(fp);
}

/*
=====
=====

```


UTILITY FUNCTIONS

```
=====
===== */
void clearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

void toLowerStr(char *str) {
    for (int i = 0; str[i]; i++) {
        str[i] = tolower((unsigned char)str[i]);
    }
}

void pause_screen() {
    printf("\nPress Enter to continue...");
    clearInputBuffer();
    getchar();
}
```

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department / faculty assessor.

Outcome	Evidence / SO Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written / oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)